

AI ASSISTED CODING

ASSIGNMENT-8

Name: Geethanjali

H.T.No:2303A51005

Task Description #1 (Username Validator – Apply AI in Authentication Context)

- Task: Use AI to generate at least 3 assert test cases for a function `is_valid_username(username)` and then implement the function using Test-Driven Development principles.

- Requirements:

- o Username length must be between 5 and 15 characters.
- o Must contain only alphabets and digits.
- o Must not start with a digit.

- o No spaces allowed. Example Assert

Test Cases:

`assert is_valid_username("User123") == True`

`assert is_valid_username("12User") == False` assert

`is_valid_username("Us er") == False`

Expected Output #1:

- Username validation logic successfully passing all AI- generated test cases.

```
79 def is_valid_username(username):
80     if len(username) < 5 or len(username) > 15:
81         return False
82     if not username.isalnum():
83         return False
84     if username[0].isdigit():
85         return False
86     if " " in username:
87         return False
88     return True
89 assert is_valid_username("User123") == True
90 assert is_valid_username("12User") == False
91 assert is_valid_username("Us er") == False
92 assert is_valid_username("User") == False
```

Task Description #2 (Even–Odd & Type Classification – Apply AI for Robust Input Handling)

- Task: Use AI to generate at least 3 assert test cases for a function `classify_value(x)` and implement it using conditional logic and loops.

- Requirements:

- o If input is an integer, classify as "Even" or "Odd".
- o If input is 0, return "Zero".
- o If input is non-numeric, return "Invalid Input".

Example Assert Test Cases:

`assert classify_value(8) == "Even" assert`

`classify_value(7) == "Odd" assert`

`classify_value("abc") == "Invalid Input"`

Expected Output #2:

- Function correctly classifying values and passing all test cases

```

95 def classify_value(x):
96     if x==0:
97         return "Zero"
98     if isinstance(x, (int, float)):
99         if x%2 == 0:
100             return "Even"
101         else:
102             return "Odd"
103     else:
104         return "Invalid Input"
105 assert classify_value(8) == "Even"
106 assert classify_value(7) == "Odd"
107 assert classify_value("abc") == "Invalid Input"

```

Task Description #3 (Palindrome Checker – Apply AI for String Normalization)

- Task: Use AI to generate at least 3 assert test cases for a function `is_palindrome(text)` and implement the function.
- Requirements:
 - o Ignore case, spaces, and punctuation.
 - o Handle edge cases such as empty strings and single characters.

Example Assert Test Cases:

`assert is_palindrome("Madam") == True assert`

`is_palindrome("A man a plan a canal Panama") == True assert`

`is_palindrome("Python") == False`

Expected Output #3:

- Function correctly identifying palindromes and passing all AI-generated tests.

```

109
110 def is_palindrome(text):
111     cleaned_text = ''.join(text.split()).lower() # Remove spaces and convert to lowercase
112     return cleaned_text == cleaned_text[::-1] # Check if the cleaned text is the same as its reverse
113 assert is_palindrome("Madam") == True
114 assert is_palindrome("A man a plan a canal Panama") == True
115 assert is_palindrome("Python") == False
116

```

Task Description #4 (Email ID Validation – Apply AI for Data Validation)

- Task: Use AI to generate at least 3 assert test cases for a function `validate_email(email)` and implement the function.

- Requirements:

o Must contain @ and . o Must not start or end

with special characters. o Should handle

invalid formats gracefully.

Example Assert Test Cases:

```
assert validate_email("user@example.com") == True
```

```
assert validate_email("userexample.com") == False
```

```
assert validate_email("@gmail.com") == False
```

Expected Output #5:

- Email validation function passing all AI-generated test cases and handling edge cases correctly.

```

119 def validate_email(email):
120     if "@" not in email or "." not in email:
121         return False
122     if email.count("@") != 1:
123         return False
124     if email.startswith("@") or email.endswith("@"):
125         return False
126     if email.startswith(".") or email.endswith("."):
127         return False
128     return True
129
130 assert validate_email("user@example.com") == True
131 assert validate_email("userexample.com") == False
132 assert validate_email("@gmail.com") == False
133

```

Task 5 (Perfect Number Checker – Test Case Design)

- Function: Check if a number is a perfect number (sum of divisors = number).

- Test Cases to Design:

o Normal case: 6 → True, 10 → False. o Edge case: 1. o Negative number case.

o Larger case: 28.

- Requirement: Validate correctness with assertions.

```

136 def perfect_number(n):
137     if n < 1:
138         return False
139     divisors_sum = sum(i for i in range(1, n) if n % i == 0)
140     return divisors_sum == n
141     assert perfect_number(6) == True
142     assert perfect_number(28) == True
143     assert perfect_number(1) == False
144     assert perfect_number(0) == False
145

```

Task 6 (Abundant Number Checker – Test Case Design)

- Function: Check if a number is abundant (sum of divisors > number).
- Test Cases to Design:
 - o Normal case: 12 → True,
 - 15 → False.
 - o Edge case: 1.
 - o Negative number case.
 - o Large case: 945.

Requirement: Validate correctness with unittest

```

147 #generate a python function abundant_number that checks if a number is abundant and validate the function using unit tests
148 def abundant_number(n):
149     if n < 1:
150         return False
151     divisors_sum = sum(i for i in range(1, n) if n % i == 0)
152     return divisors_sum > n
153 # Unit tests for the abundant_number function
154 import unittest
155 class TestAbundantNumber(unittest.TestCase):
156     def test_abundant_number(self):
157         self.assertTrue(abundant_number(12)) # 1 + 2 + 3 + 4 + 6 = 16 > 12
158     def test_non_abundant_number(self):
159         self.assertFalse(abundant_number(28)) # 1 + 2 + 4 + 7 + 14 = 28 == 28
160     def test_edge_cases(self):
161         self.assertFalse(abundant_number(1)) # No proper divisors
162         self.assertFalse(abundant_number(0)) # Not a valid input
163     def test_negative_number(self):
164         self.assertFalse(abundant_number(-5)) # Not a valid input
165 if __name__ == "__main__":
166     unittest.main() # Run the unit tests
167

```

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS
<pre> PS C:\Users\gudah> & C:/Python314/python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_8.py Ran 4 tests in 0.001s OK PS C:\Users\gudah> </pre>				

Task 7 (Deficient Number Checker – Test Case Design)

- Function: Check if a number is deficient (sum of divisors < number).
- Test Cases to Design:

o Normal case: $8 \rightarrow \text{True}$,

$12 \rightarrow \text{False}$. o Edge case: 1.

o Negative number case. o

Large case: 546.

Requirement: Validate correctness with pytest.

```
170 #generate a python function number is deficient and validate the function using pytest
171 def deficient_number(n):
172     if n < 1:
173         return False
174     divisors_sum = sum(i for i in range(1, n) if n % i == 0)
175     return divisors_sum < n
176 # Unit tests for the deficient_number function using pytest
177 def test_deficient_number():
178     assert deficient_number(8) == True # 1 + 2 + 4 = 7 < 8
179 def test_non_deficient_number():
180     assert deficient_number(12) == False # 1 + 2 + 3 + 4 + 6 = 16 > 12
181 def test_edge_cases():
182     assert deficient_number(1) == True
183     assert deficient_number(0) == False # Not a valid input
184 def test_negative_number():
185     assert deficient_number(-5) == False # Not a valid input
186
```

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS
PS C:\Users\gudah\OneDrive\Documents\AIAC> python -m pytest Lab_assignment_8.py				
PS C:\Users\gudah\OneDrive\Documents\AIAC> & C:/Python314/python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_8.py				
PS C:\Users\gudah\OneDrive\Documents\AIAC> python -m pytest Lab_assignment_8.py				
===== test session starts =====				
platform win32 -- Python 3.14.2, pytest-9.0.2, pluggy-1.6.0				
rootdir: C:\Users\gudah\OneDrive\Documents\AIAC				
plugins: anyio-4.12.1				
collected 4 items				
Lab_assignment_8.py				
===== 4 passed in 0.09s =====				

Task 8 : Write a function LeapYearChecker and validate its implementation using 10 pytest test cases

```
189 #Write a function LeapYearChecker and validate its implementation using 10 pytest test cases
190 def is_leap_year(year):
191     if year % 4 == 0:
192         if year % 100 == 0:
193             return year % 400 == 0
194         return True
195     return False
196
197 def test_leap_year():
198     assert is_leap_year(2024) == True # Divisible by 4, not by 100
199     assert is_leap_year(2023) == False # Not divisible by 4
200     assert is_leap_year(1900) == False # Divisible by 100 but not by 400
201     assert is_leap_year(2000) == True # Divisible by 400
202     assert is_leap_year(2100) == False # Divisible by 100 but not by 400
203     assert is_leap_year(2400) == True # Divisible by 4, and also divisible by 400
204     assert is_leap_year(1656) == True # Divisible by 4, not by 100
205     assert is_leap_year(1776) == True # Divisible by 4, not by 100
206     assert is_leap_year(1888) == True # Divisible by 4, not by 100
207     assert is_leap_year(1999) == False # Not divisible by 4
```

Task 9 :Write a function SumOfDigits and validate its implementation using 7 pytest test cases.


```

210 #Write a function SumOfDigits and validate its implementation using 7 pytest test cases.
211 def sum_of_digits(n):
212     if n < 0:
213         raise ValueError("Input must be a non-negative integer.")
214     return sum(int(digit) for digit in str(n))
215 def test_sum_of_digits():
216     assert sum_of_digits(123) == 6    # 1 + 2 + 3 = 6
217     assert sum_of_digits(0) == 0      # Sum of digits in 0 is 0
218     assert sum_of_digits(999) == 27   # 9 + 9 + 9 = 27
219     assert sum_of_digits(4567) == 22  # 4 + 5 + 6 + 7 = 22
220     assert sum_of_digits(1001) == 2   # 1 + 0 + 0 + 1 = 2
221     assert sum_of_digits(12345) == 15 # 1 + 2 + 3 + 4 + 5 = 15
222     try:
223         sum_of_digits(-123)    # Should raise ValueError for negative input
224     except ValueError:
225         pass
226

```

Task 10 : Write a function SortNumbers (implement bubble sort) and validate its implementation using 25 pytest test cases.

```

#Write a function SortNumbers (implement bubble sort) and validate its implementation using 25 pytest test cases.
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
def test_bubble_sort():
    assert bubble_sort([64, 34, 25, 12, 22, 11, 90]) == [11, 12, 22, 25, 34, 64, 90]
    assert bubble_sort([5, 1, 4, 2, 8]) == [1, 2, 4, 5, 8]
    assert bubble_sort([]) == []
    assert bubble_sort([1]) == [1]
    assert bubble_sort([2, 1]) == [1, 2]
    assert bubble_sort([3, 3, 3]) == [3, 3, 3]
    assert bubble_sort([-1, -5, -3]) == [-5, -3, -1]
    assert bubble_sort([0]) == [0]
    assert bubble_sort([10, -10]) == [-10, 10]
    assert bubble_sort([1.5, 0.5, -0.5]) == [-0.5, 0.5, 1.5]
    assert bubble_sort([1000000, -1000000]) == [-1000000, 1000000]
    assert bubble_sort([9, -9]) == [-9, 9]
    assert bubble_sort([2.71828]) == [2.71828]
    assert bubble_sort([-3.14]) == [-3.14]
    assert bubble_sort([1e-10]) == [1e-10]
    assert bubble_sort([-1e-10]) == [-1e-10]
    assert bubble_sort([123456789]) == [123456789]

```

```

    assert bubble_sort([1e-10]) == [1e-10]
    assert bubble_sort([-1e-10]) == [-1e-10]
    assert bubble_sort([123456789]) == [123456789]
    assert bubble_sort([-123456789]) == [-123456789]
    assert bubble_sort([0.00001]) == [0.00001]
    assert bubble_sort([-0.00001]) == [-0.00001]

```

MS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS
<pre> \Users\gudah\OneDrive\Documents\AIAC> python -m pytest Lab_assignment_8.py ===== test session starts ===== platform win32 -- Python 3.14.2, pytest-9.0.2, pluggy-1.6.0 platform win32 -- Python 3.14.2, pytest-9.0.2, pluggy-1.6.0 rootdir: C:\Users\gudah\OneDrive\Documents\AIAC plugins: anyio-4.12.1 collected 1 item Lab_assignment_8.py . ===== 1 passed in 0.11s ===== \Users\gudah\OneDrive\Documents\AIAC> </pre>				

Task 11 : Write a function ReverseString and validate its implementation using 5 unittest test cases

```
260 #Write a function ReverseString and validate its implementation using 5 unittest test cases
261 def reverse_string(s):
262     | return s[::-1]
263 import unittest
264 class TestReverseString(unittest.TestCase):
265     | def test_reverse_string(self):
266     |     self.assertEqual(reverse_string("hello"), "olleh")
267     |     self.assertEqual(reverse_string("Python"), "nohtyp")
268     |     self.assertEqual(reverse_string(""), "")
269     |     self.assertEqual(reverse_string("a"), "a")
270     |     self.assertEqual(reverse_string("12345"), "54321")
271 if __name__ == "__main__":
272     | unittest.main() # Run the unit tests
273
```

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS
----------	--------	---------------	----------	-------

```
● PS C:\Users\gudah> & C:/Python314/python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_8.py
● PS C:\Users\gudah> & C:/Python314/python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_8.py
*
-----
Ran 1 test in 0.000s

OK
○ PS C:\Users\gudah>
```

Task 12 : Write a function AnagramChecker and validate its implementation using 10 unittest test cases.

```
276 #Write a function AnagramChecker and validate its implementation using 10 unittest test cases.
277 def are_anagrams(str1, str2):
278     | return sorted(str1.replace(" ", "").lower()) == sorted(str2.replace(" ", "").lower())
279 import unittest
280 class TestAnagramChecker(unittest.TestCase):
281     | def test_anagrams(self):
282     |     self.assertTrue(are_anagrams("listen", "silent"))
283     |     self.assertTrue(are_anagrams("Triangle", "Integral"))
284     |     self.assertTrue(are_anagrams("Dormitory", "Dirty Room"))
285     |     self.assertFalse(are_anagrams("Hello", "World"))
286     |     self.assertFalse(are_anagrams("Python", "Java"))
287     |     self.assertTrue(are_anagrams("A gentleman", "Elegant man"))
288     |     self.assertFalse(are_anagrams("Test", "Best"))
289     |     self.assertTrue(are_anagrams("Clint Eastwood", "Old West Action"))
290     |     self.assertTrue(are_anagrams("School master", "The classroom"))
291 if __name__ == "__main__":
292     | unittest.main() # Run the unit tests
293
```

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS
----------	--------	---------------	----------	-------

```
● PS C:\Users\gudah> & C:/Python314/python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_8.py
● PS C:\Users\gudah> & C:/Python314/python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_8.py
*
-----
Ran 1 test in 0.000s

OK
○ PS C:\Users\gudah>
```

Task 13 : Write a function `ArmstrongChecker` and validate its implementation using 8 unittest test cases.

```
297 def is_armstrong_number(n):
298     if n < 0:
299         return False
300     num_str = str(n)
301     num_digits = len(num_str)
302     armstrong_sum = sum(int(digit) ** num_digits for digit in num_str)
303     return armstrong_sum == n
304
305 class TestArmstrongChecker(unittest.TestCase):
306     def test_armstrong_numbers(self):
307         self.assertTrue(is_armstrong_number(153)) # 1^3 + 5^3 + 3^3 = 153
308         self.assertTrue(is_armstrong_number(370)) # 3^3 + 7^3 + 0^3 = 370
309         self.assertTrue(is_armstrong_number(9474)) # 9^4 + 4^4 + 7^4 + 4^4 = 9474
310         self.assertFalse(is_armstrong_number(123)) # Not an Armstrong number
311         self.assertFalse(is_armstrong_number(-153)) #negative numbers cannot be Armstrong numbers
312         self.assertTrue(is_armstrong_number(1)) # 1^1 = 1
313         self.assertFalse(is_armstrong_number(10)) # Not an Armstrong number
314
315 if __name__ == "__main__":
316     unittest.main() # Run the unit tests
317
```

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS
<pre>PS C:\Users\gudah> & C:/Python314/python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_8.py . ----- Ran 1 test in 0.000s OK PS C:\Users\gudah></pre>				